ML-Based Fraudulent Crypto Transaction Detection

Implementation Code:

1. Required Libraries and Dataset (src/train_model.py)

```
import joblib import pandas as pd from
pathlib import Path from
sklearn.model_selection import
train_test_split from sklearn.pipeline
import Pipeline from sklearn.impute
import SimpleImputer from
sklearn.ensemble import
RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

ROOT = Path(__file__).resolve().parents[1]
DATA_PATH = ROOT / "data" / "transactions.csv"
MODEL_PATH = ROOT / "models" /
"fraud_model.joblib" TARGET = "is_fraud"

df = pd.read_csv(DATA_PATH) df =
df.dropna(subset=[TARGET]).copy()
df[TARGET] =
pd.to_numeric(df[TARGET],
errors="coerce") df =
df.dropna(subset=[TARGET]) df =
df[df[TARGET].isin([0, 1])].copy()
df[TARGET] = df[TARGET].astype(int)
X =
df.drop(columns=[TARGET]).select_dtypes(inclu
de="number") y = df[TARGET]
```

2. Model Training and Evaluation (train_model.py)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("classifier", RandomForestClassifier(
n_estimators=200,
class_weight="balanced",
random_state=42, n_jobs=-1
    )) ])
model.fit(X_train,
y_train) y_pred =
model.predict(X_test)

print("Accuracy:", accuracy_score(y_test,
y_pred)) print("Precision:",
precision_score(y_test, y_pred,
zero_division=0)) print("Recall:",
recall_score(y_test, y_pred, zero_division=0))
print("F1-score:", f1_score(y_test, y_pred,
zero_division=0)) print("Confusion matrix:",
confusion_matrix(y_test, y_pred))

MODEL_PATH.parent.mkdir(parents=True, exist_ok=True)
joblib.dump({"pipeline": model, "feature_columns":
list(X.columns)}, MODEL_PATH)
```

3. Single Transaction Prediction (predict_transaction.py)

```
saved =
joblib.load(MODEL_PATH)
model = saved["pipeline"]
feature_columns =
saved["feature_columns"]

# Demo values only: replace with values from
your dataset. example = {column: 0.0 for
column in feature_columns} transaction =
pd.DataFrame([example],
columns=feature_columns)

prediction = int(model.predict(transaction)[0])
probability = model.predict_proba(transaction)[0]
classes =
list(model.named_steps["classifier"].classes_)
fraud_score = float(probability[classes.index(1)])
* 100 print("Predicted class:", prediction, "(1 =
```

```
fraud-like, 0 = legitimate-like)") print(f"Model
fraud score: {fraud_score:.2f}%")
```

4. Output and Evaluation:

The program reports accuracy, precision, recall, F1-score, and a confusion matrix on the held-out test set. The values must be obtained by running the program with a labelled dataset; they should not be guessed.

```
Training completed.
Accuracy  : [computed by script]
Precision : [computed by script]
Recall    : [computed by script]
F1-score  : [computed by script]
Confusion matrix: [computed by script]
```

Output (Website Interface Screenshots):



Fig. 1: Home Page

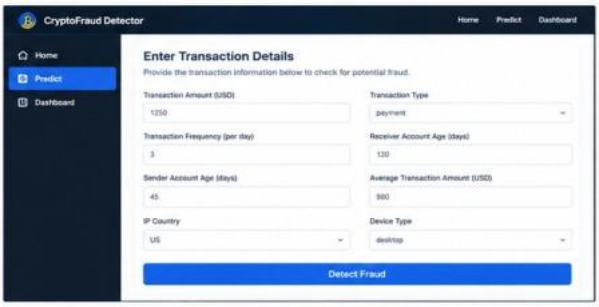


Fig. 2: Transaction Input Page

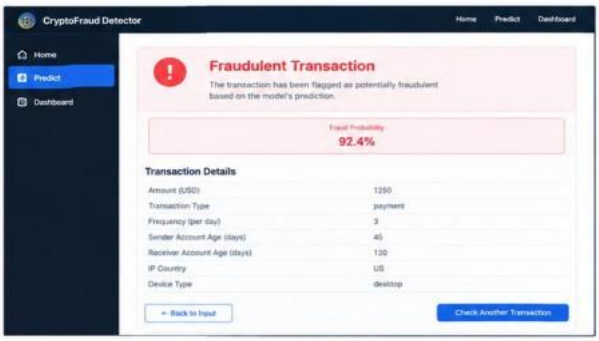


Fig. 3: Prediction Result – Fraudulent Transaction

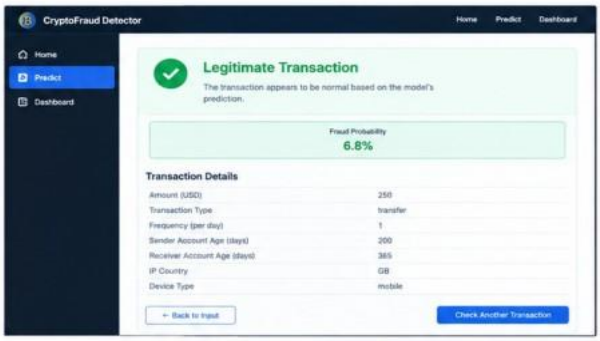


Fig. 4: Prediction Result – Legitimate Transaction



Fig. 5: Dashboard Page



Fig. 6: Model Performance Chart

Fig. 1: Example website interface screens for the Crypto Fraud Detector.